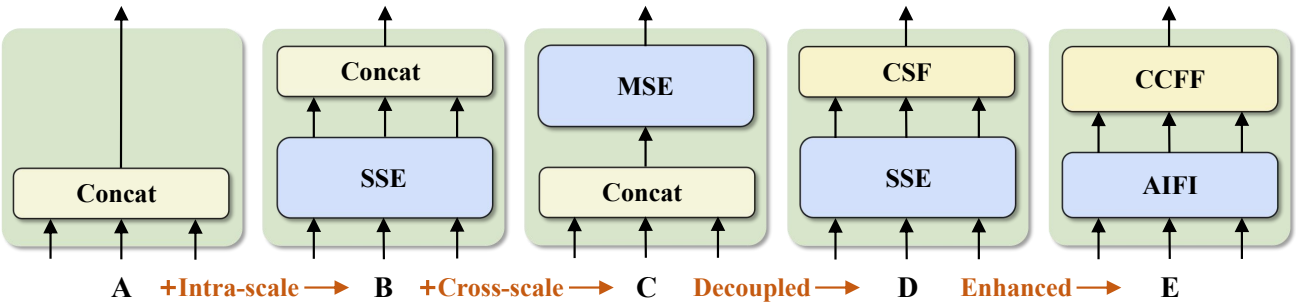

Figure 3. The encoder structure for each variant. **SSE** represents the single-scale Transformer encoder, **MSE** represents the multi-scale Transformer encoder, and **CSF** represents cross-scale fusion. **AIFI** and **CCFF** are the two modules designed into our hybrid encoder.

on T4 GPU with TensorRT FP16. According to the results (*cf. Table* 2), we conclude that *anchor-free detectors outperform anchor-based detectors with equivalent accuracy for YOLO detectors because the former require less NMS time than the latter*. The reason is that anchor-based detectors produce more prediction boxes than anchor-free detectors (three times more in our tested detectors).

4. The Real-time DETR

4.1. Model Overview

RT-DETR consists of a backbone, an efficient hybrid encoder, and a Transformer decoder with auxiliary prediction heads. The overview of RT-DETR is illustrated in Figure 4. Specifically, we feed the features from the last three stages of the backbone $\{\mathcal{S}_3, \mathcal{S}_4, \mathcal{S}_5\}$ into the encoder. The efficient hybrid encoder transforms multi-scale features into a sequence of image features through intra-scale feature interaction and cross-scale feature fusion (*cf.* Sec. 4.2). Subsequently, the uncertainty-minimal query selection is employed to select a fixed number of encoder features to serve as initial object queries for the decoder (*cf.* Sec. 4.3). Finally, the decoder with auxiliary prediction heads iteratively optimizes object queries to generate categories and boxes.

4.2. Efficient Hybrid Encoder

Computational bottleneck analysis. The introduction of multi-scale features accelerates training convergence and improves performance [45]. However, although the deformable attention reduces the computational cost, the sharply increased sequence length still causes the encoder to become the computational bottleneck. As reported in Lin *et al*. [19], the encoder accounts for 49% of the GFLOPs but contributes only 11% of the AP in Deformable-DETR. To overcome this bottleneck, we first analyze the computational redundancy present in the multi-scale Transformer encoder. Intuitively, high-level features that contain rich semantic information about objects are extracted from low-level features, making it redundant to perform feature interaction on the concatenated multi-scale features. Therefore, we design a set of variants with different types of the encoder to prove that the simultaneous intra-scale and cross-scale feature interaction is inefficient, Figure 3. Specially, we use DINO-Deformable-R50 with the smaller size data reader and lighter decoder used in RT-DETR for experiments and first remove the multi-scale Transformer encoder in DINO-Deformable-R50 as variant A. Then, different types of the encoder are inserted to produce a series of variants based on A, elaborated as follows (Detailed indicators of each variant are referred to in Table 3):

- A $\rightarrow$ B: Variant B inserts a single-scale Transformer encoder into A, which uses one layer of Transformer block. The multi-scale features share the encoder for intra-scale feature interaction and then concatenate as output.
- B $\rightarrow$ C: Variant C introduces cross-scale feature fusion based on B and feeds the concatenated features into the multi-scale Transformer encoder to perform simultaneous intra-scale and cross-scale feature interaction.
- C $\rightarrow$ D: Variant D decouples intra-scale interaction and cross-scale fusion by utilizing the single-scale Transformer encoder for the former and a PANet-style [22] structure for the latter.
- D $\rightarrow$ E: Variant E enhances the intra-scale interaction and cross-scale fusion based on D, adopting an efficient hybrid encoder designed by us.

Hybrid design. Based on the above analysis, we rethink the structure of the encoder and propose an *efficient hybrid encoder*, consisting of two modules, namely the Attention-based Intra-scale Feature Interaction (AIFI) and the CNN-based Cross-scale Feature Fusion (CCFF). Specifically, AIFI further reduces the computational cost based on variant D by performing the intra-scale interaction only on $\mathcal{S}_5$ with the single-scale Transformer encoder. The reason is that applying the self-attention operation to high-level features with richer semantic concepts captures the connection between conceptual entities, which facilitates the localization and recognition of objects by subsequent modules. However, the intra-scale interactions of lower-level features are unnecessary due to the lack of semantic concepts and the risk of duplication and confusion with high-level feature interactions. To verify this opinion, we perform the intra-scale interaction only on $\mathcal{S}_5$ in variant D, and the experimental results are reported in Table 3 (see row $\text{D}_{\mathcal{S}_5}$). Compared to

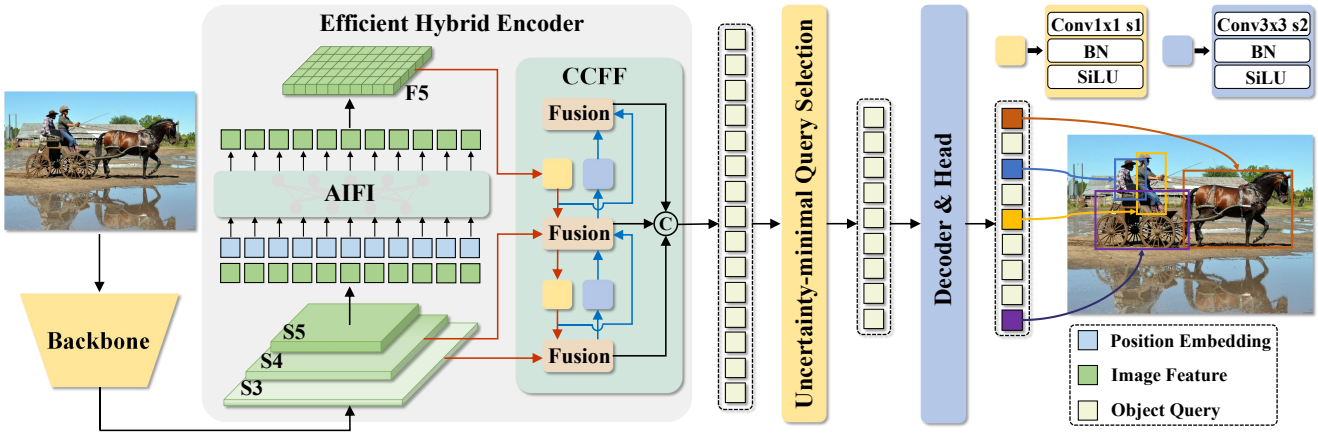

Figure 4. Overview of RT-DETR. We feed the features from the last three stages of the backbone into the encoder. The efficient hybrid encoder transforms multi-scale features into a sequence of image features through the Attention-based Intra-scale Feature Interaction (AIFI) and the CNN-based Cross-scale Feature Fusion (CCFF). Then, the uncertainty-minimal query selection selects a fixed number of encoder features to serve as initial object queries for the decoder. Finally, the decoder with auxiliary prediction heads iteratively optimizes object queries to generate categories and boxes.

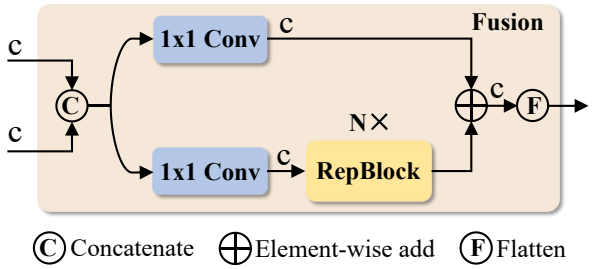

Figure 5. The fusion block in CCFF.

D, $D_{\mathcal{S}_5}$ not only significantly reduces latency (35% faster), but also improves accuracy (0.4% AP higher). CCFF is optimized based on the cross-scale fusion module, which inserts several fusion blocks consisting of convolutional layers into the fusion path. The role of the fusion block is to fuse two adjacent scale features into a new feature, and its structure is illustrated in Figure 5. The fusion block contains two 1×1 convolutions to adjust the number of channels, N *RepBlocks* composed of RepConv [8] are used for feature fusion, and the two-path outputs are fused by element-wise add. We formulate the calculation of the hybrid encoder as:

$$\begin{aligned} \mathcal{Q} = \mathcal{K} = \mathcal{V} &= \texttt{Flatten}(\mathcal{S}_5), \\ \mathcal{F}_5 &= \texttt{Reshape}(\texttt{AIFI}(\mathcal{Q}, \mathcal{K}, \mathcal{V})), \\ \mathcal{O} &= \texttt{CCFF}(\{\mathcal{S}_3, \mathcal{S}_4, \mathcal{F}_5\}), \end{aligned} \tag{1}$$

where `Reshape` represents restoring the shape of the flattened feature to the same shape as $\mathcal{S}_5$.

4.3. Uncertainty-minimal Query Selection

To reduce the difficulty of optimizing object queries in DETR, several subsequent works [42, 44, 45] propose query selection schemes, which have in common that they use the confidence score to select the top K features from the encoder to initialize object queries (or just position queries). The confidence score represents the likelihood that the feature includes foreground objects. Nevertheless, the detector are required to simultaneously model the category and location of objects, both of which determine the quality of the features. Hence, the performance score of the feature is a latent variable that is jointly correlated with both classification and localization. Based on the analysis, the current query selection lead to a considerable level of uncertainty in the selected features, resulting in sub-optimal initialization for the decoder and hindering the performance of the detector.

To address this problem, we propose the uncertainty minimal query selection scheme, which explicitly constructs and optimizes the epistemic uncertainty to model the joint latent variable of encoder features, thereby providing high-quality queries for the decoder. Specifically, the feature uncertainty $\mathcal{U}$ is defined as the discrepancy between the predicted distributions of localization $\mathcal{P}$ and classification $\mathcal{C}$ in Eq. (2). To minimize the uncertainty of the queries, we integrate the uncertainty into the loss function for the gradient-based optimization in Eq. (3).

$$\mathcal{U}(\hat{\mathbf{x}}) = \|\mathcal{P}(\hat{\mathbf{x}}) - \mathcal{C}(\hat{\mathbf{x}})\|, \hat{\mathbf{x}} \in \mathbb{R}^D \tag{2}$$

$$\mathcal{L}(\hat{\mathbf{x}}, \hat{\mathbf{y}}, \mathbf{y}) = \mathcal{L}_{box}(\hat{\mathbf{b}}, \mathbf{b}) + \mathcal{L}_{cls}(\mathcal{U}(\hat{\mathbf{x}}), \hat{\mathbf{c}}, \mathbf{c}) \tag{3}$$

where $\hat{\mathbf{y}}$ and $\mathbf{y}$ denote the prediction and ground truth, $\hat{\mathbf{y}} = \{\hat{\mathbf{c}}, \hat{\mathbf{b}}\}$, $\hat{\mathbf{c}}$ and $\hat{\mathbf{b}}$ represent the category and bounding box respectively, $\hat{\mathbf{x}}$ represent the encoder feature.

Effectiveness analysis. To analyze the effectiveness of the uncertainty-minimal query selection, we visualize the classification scores and IoU scores of the selected features on COCO `val2017`, Figure 6. We draw the scatterplot with classification scores greater than 0.5. The purple and green dots represent the selected features from the model trained with uncertainty-minimal query selection and vanilla query

Model	Backbone	#Epochs	#Params (M)	GFLOPs	$FPS_{bs=1}$	AP^{val}	AP^{val}_{50}	AP^{val}_{75}	AP^{val}_{S}	AP^{val}_{M}	AP^{val}_{L}
Real-time Object Detectors											
YOLOv5-L [11]	-	300	46	109	54	49.0	67.3	-	-	-	-
YOLOv5-X [11]	-	300	86	205	43	50.7	68.9	-	-	-	-
PPYOLOE-L [40]	-	300	52	110	94	51.4	68.9	55.6	31.4	55.3	66.1
PPYOLOE-X [40]	-	300	98	206	60	52.3	69.9	56.5	33.3	56.3	66.4
YOLOv6-L [16]	-	300	59	150	99	52.8	70.3	57.7	34.4	58.1	70.1
YOLOv7-L [38]	-	300	36	104	55	51.2	69.7	55.5	35.2	55.9	66.7
YOLOv7-X [38]	-	300	71	189	45	52.9	71.1	57.4	36.9	57.7	68.6
YOLOv8-L [12]	-	-	43	165	71	52.9	69.8	57.5	35.3	58.3	69.8
YOLOv8-X [12]	-	-	68	257	50	53.9	71.0	58.7	35.7	59.3	70.7
End-to-end Object Detectors											
DETR-DC5 [4]	R50	500	41	187	-	43.3	63.1	45.9	22.5	47.3	61.1
DETR-DC5 [4]	R101	500	60	253	-	44.9	64.7	47.7	23.7	49.5	62.3
Anchor-DETR-DC5 [39]	R50	50	39	172	-	44.2	64.7	47.5	24.7	48.2	60.6
Anchor-DETR-DC5 [39]	R101	50	-	-	-	45.1	65.7	48.8	25.8	49.4	61.6
Conditional-DETR-DC5 [27]	R50	108	44	195	-	45.1	65.4	48.5	25.3	49.0	62.2
Conditional-DETR-DC5 [27]	R101	108	63	262	-	45.9	66.8	49.5	27.2	50.3	63.3
Efficient-DETR [42]	R50	36	35	210	-	45.1	63.1	49.1	28.3	48.4	59.0
Efficient-DETR [42]	R101	36	54	289	-	45.7	64.1	49.5	28.2	49.1	60.2
SMCA-DETR [9]	R50	108	40	152	-	45.6	65.5	49.1	25.9	49.3	62.6
SMCA-DETR [9]	R101	108	58	218	-	46.3	66.6	50.2	27.2	50.5	63.2
Deformable-DETR [45]	R50	50	40	173	-	46.2	65.2	50.0	28.8	49.2	61.7
DAB-Deformable-DETR [23]	R50	50	48	195	-	46.9	66.0	50.8	30.1	50.4	62.5
DAB-Deformable-DETR++ [23]	R50	50	47	-	-	48.7	67.2	53.0	31.4	51.6	63.9
DN-Deformable-DETR [17]	R50	50	48	195	-	48.6	67.4	52.7	31.0	52.0	63.7
DN-Deformable-DETR++ [17]	R50	50	47	-	-	49.5	67.6	53.8	31.3	52.6	65.4
DINO-Deformable-DETR [44]	R50	36	47	279	5	50.9	69.0	55.3	34.6	54.1	64.6
Real-time End-to-end Object Detector (ours)											
RT-DETR	R50	72	42	136	**108**	**53.1**	**71.3**	**57.7**	34.8	58.0	70.0
RT-DETR	R101	72	76	259	**74**	**54.3**	**72.7**	58.6	36.0	58.8	**72.1**

Table 2. Comparison with SOTA (only L and X models of YOLO detectors, see Appendix for the comparison with S and M models). We do not test the speed of other DETRs, except for DINO-Deformable-DETR [44] for comparison, as they are not real-time detectors. Our RT-DETR outperforms the state-of-the-art YOLO detectors and DETRs in both speed and accuracy.

We test the speed of DINO-Deformable-DETR [44] according to the settings of the corresponding accuracy taken on COCO `val2017` for comparison, *i.e.*, the speed is tested with TensorRT FP16 and the input size is (800, 1333). Table 2 shows that RT-DETR outperforms all DETRs with the same backbone in both speed and accuracy. Compared to DINO-Deformable-DETR-R50, RT-DETR-R50 improves the accuracy by 2.2% AP and the speed by 21 times (108 FPS vs 5 FPS), both of which are significantly improved.

5.2. Ablation Study on Hybrid Encoder

We evaluate the indicators of the variants designed in Sec. 4.2, including AP (trained with $1\times$ configuration), the number of parameters, and the latency, Table 3. Compared to baseline A, variant B improves accuracy by 1.9% AP and increases the latency by 54%. This proves that the intra-scale feature interaction is significant, but the single-scale Transformer encoder is computationally expensive. Variant C delivers a 0.7% AP improvement over B and increases the latency by 20%. This shows that the cross-scale feature fusion is also necessary but the multi-scale Transformer encoder requires higher computational cost. Variant D delivers a 0.8% AP improvement over C, but reduces latency by 8%, suggesting that decoupling intra-scale interaction and cross-scale fusion not only reduces computational cost but also improves accuracy. Compared to variant D, $D_{\mathcal{S}_5}$ reduces the latency by 35% but delivers 0.4% AP improvement, demonstrating that intra-scale interactions of lower-level features are not required. Finally, variant E delivers 1.5% AP improvement over D. Despite a 20% increase in the number of parameters, the latency is reduced by 24%, making the encoder more efficient. This shows that our hybrid encoder achieves a better trade-off between speed and accuracy.

5.3. Ablation Study on Query Selection

We conduct an ablation study on uncertainty-minimal query selection, and the results are reported on RT-DETR-R50 with $1\times$ configuration, Table 4. The query selection in RT-DETR selects the top K ($K = 300$) encoder features according to the classification scores as the content queries, and the prediction boxes corresponding to the selected features are used as initial position queries. We compare the encoder features selected by the two query selection schemes on COCO `val2017` and calculate the proportions of classi-